CSharPN

A Code-First Framework for Coloured Petri Net Modelling with C# as Inscription Language

Simon Tjell

Independent Researcher · simon@simplesystemer.dk

PNSE'26 · Petri Nets and Software Engineering · June 2026

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

- **CPN Tools / CPN IDE** → Standard ML inscriptions

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

- **CPN Tools / CPN IDE** → Standard ML inscriptions
- Dedicated **graphical editor** — not their IDE

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

- **CPN Tools / CPN IDE** → Standard ML inscriptions
- Dedicated **graphical editor** — not their IDE
- Binary / XML model files — resist diff & review

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

- **CPN Tools / CPN IDE** → Standard ML inscriptions
- Dedicated **graphical editor** — not their IDE
- Binary / XML model files — resist diff & review
- Model and implementation live in **two different worlds**

The gap

Coloured Petri Nets are great for concurrent systems, protocols, business processes.

Mainstream developers face a hurdle with existing tools:

- **CPN Tools / CPN IDE** → Standard ML inscriptions
- Dedicated **graphical editor** — not their IDE
- Binary / XML model files — resist diff & review
- Model and implementation live in **two different worlds**

> *What if a CPN model were just a C# file in your repo?*

CPNs in 30 seconds

Two extensions used in this talk:

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)

Two extensions used in this talk:

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)
- **Transitions** fire under a **binding** of arc variables

Two extensions used in this talk:

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)
- **Transitions** fire under a **binding** of arc variables
- Enabled when input arcs are satisfied and the **guard** holds

Two extensions used in this talk:

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)
- **Transitions** fire under a **binding** of arc variables
- Enabled when input arcs are satisfied and the **guard** holds
- Firing consumes input tokens, produces output tokens

Two extensions used in this talk:

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)
- **Transitions** fire under a **binding** of arc variables
- Enabled when input arcs are satisfied and the **guard** holds
- Firing consumes input tokens, produces output tokens

Two extensions used in this talk:

- **Timed CPNs**: tokens carry timestamps, global clock advances

CPNs in 30 seconds

- **Places** hold multisets of **typed tokens** (colour set)
- **Transitions** fire under a **binding** of arc variables
- Enabled when input arcs are satisfied and the **guard** holds
- Firing consumes input tokens, produces output tokens

Two extensions used in this talk:

- **Timed CPNs**: tokens carry timestamps, global clock advances
- **Hierarchical CPNs**: substitution transitions + port places (In, Out, I/O)

CSharPN

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

- C# is **both** the host language **and** the inscription language

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

- C# is **both** the host language **and** the inscription language
- A model is an ordinary class inheriting from `CpnModel`

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

- C# is **both** the host language **and** the inscription language
- A model is an ordinary class inheriting from `CpnModel`
- Colour sets → C# types | arcs & guards → C# lambdas

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

- C# is **both** the host language **and** the inscription language
- A model is an ordinary class inheriting from `CpnModel`
- Colour sets → C# types | arcs & guards → C# lambdas
- Full **IntelliSense**, **compile-time type checking**, .NET ecosystem

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

- C# is **both** the host language **and** the inscription language
- A model is an ordinary class inheriting from `CpnModel`
- Colour sets → C# types | arcs & guards → C# lambdas
- Full **IntelliSense**, **compile-time type checking**, .NET ecosystem
- Untimed, **timed**, and **hierarchical** CPNs in one API

Open source (MIT) — github.com/simontjell/CSharPN.public

CSharPN

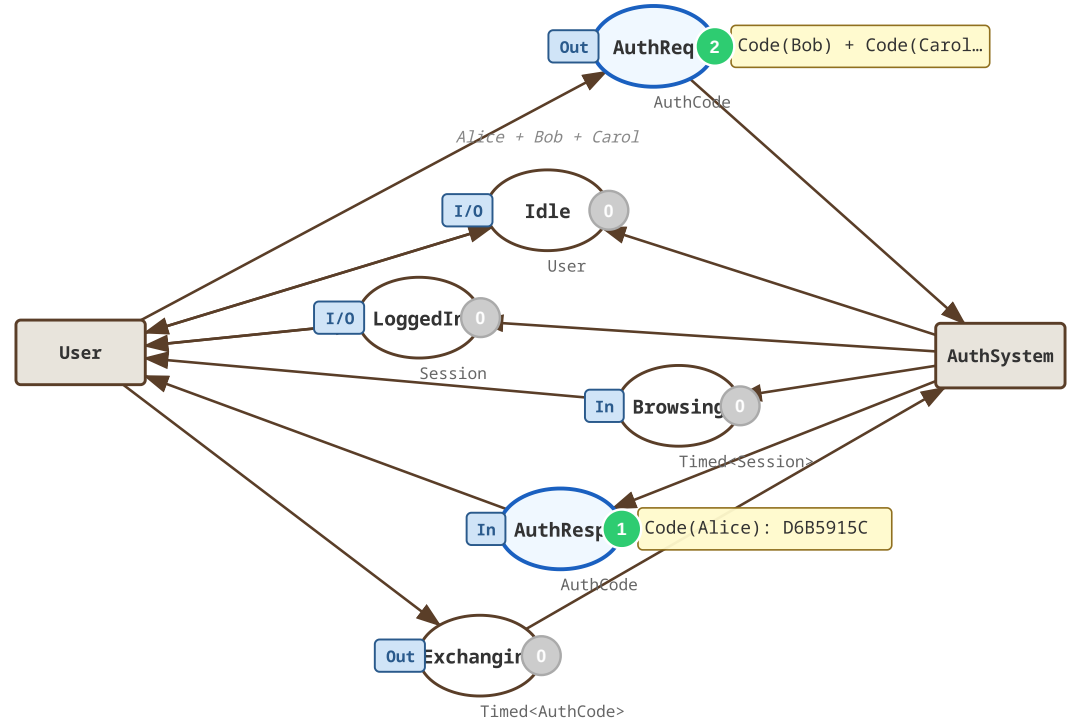
- C# is **both** the host language **and** the inscription language
- A model is an ordinary class inheriting from `CpnModel`
- Colour sets → C# types | arcs & guards → C# lambdas
- Full **IntelliSense**, **compile-time type checking**, .NET ecosystem
- Untimed, **timed**, and **hierarchical** CPNs in one API
- Interactive **Blazor visualiser** embedded in VS Code

Open source (MIT) — github.com/simontjell/CSharPN.public

Running example: OAuth2 authoris

Exercises all three features:

Top page: shared places link the two substitution transitions.

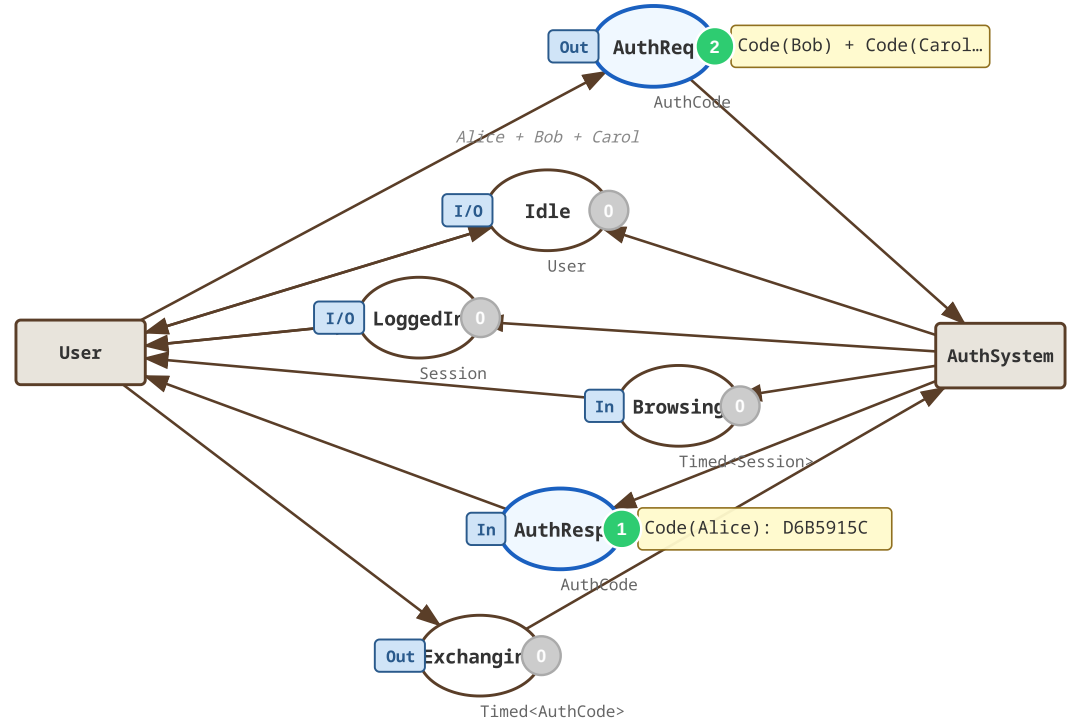


Running example: OAuth2 authoris

Exercises all three features:

- Three concurrent users

Top page: shared places link the two substitution transitions.

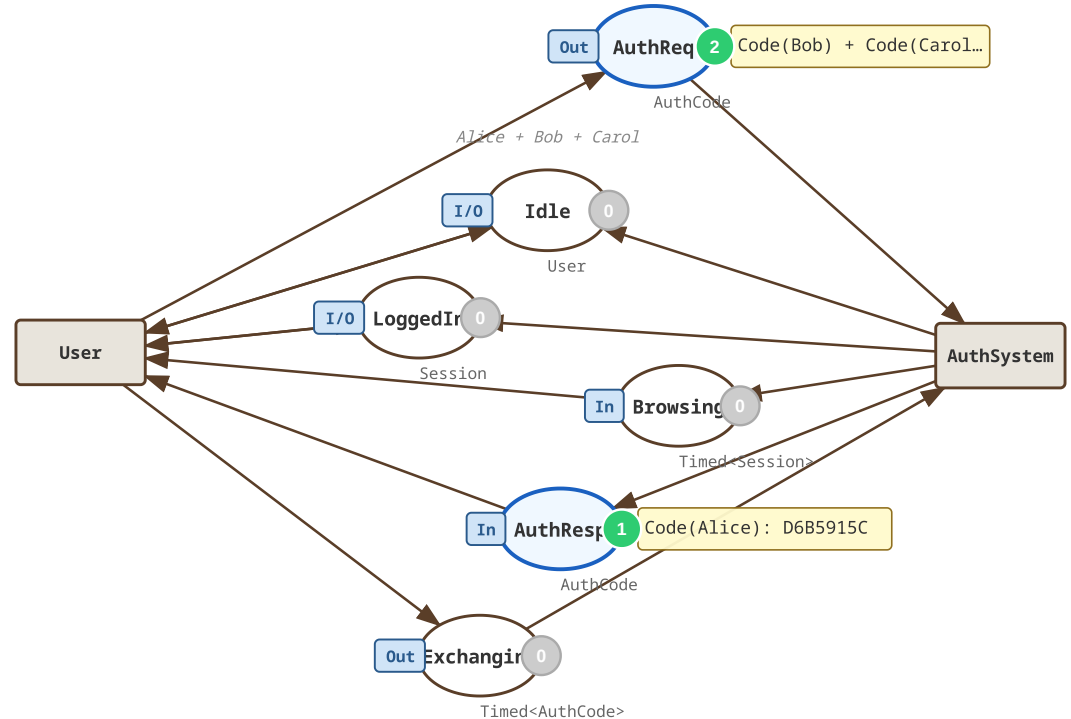


Running example: OAuth2 authoris

Exercises all three features:

- Three concurrent users
- Shared places as channels

Top page: shared places link the two substitution transitions.

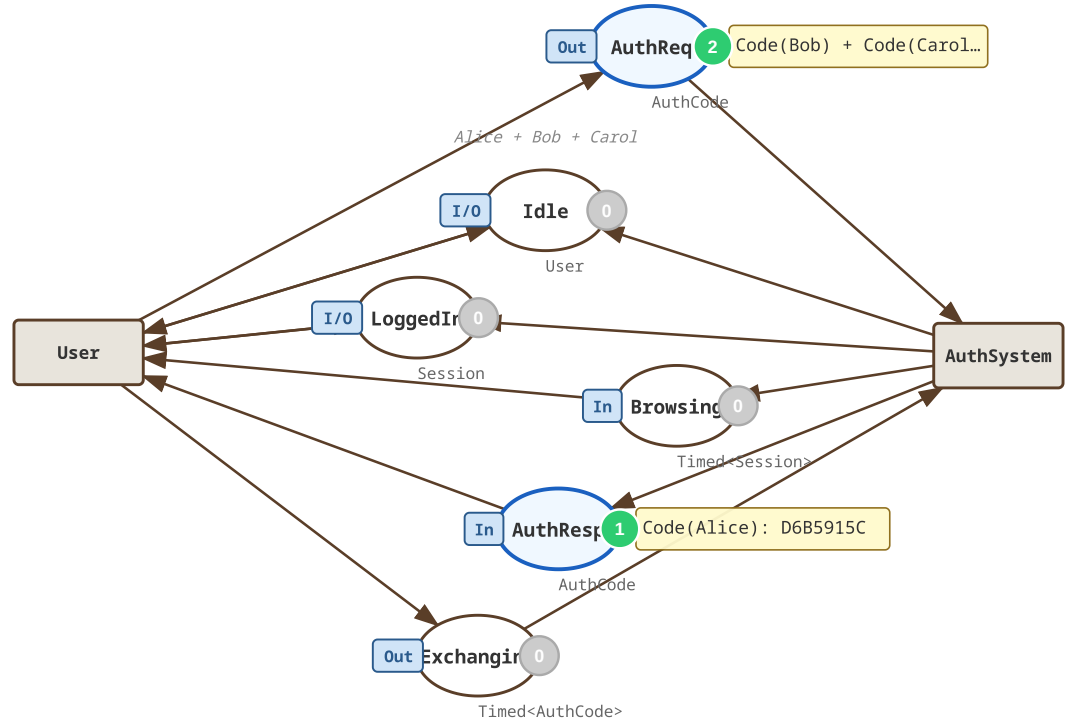


Running example: OAuth2 authoris

Exercises all three features:

- Three concurrent users
- Shared places as channels
- Timed auth, exchange, expiry

Top page: shared places link the two substitution transitions.

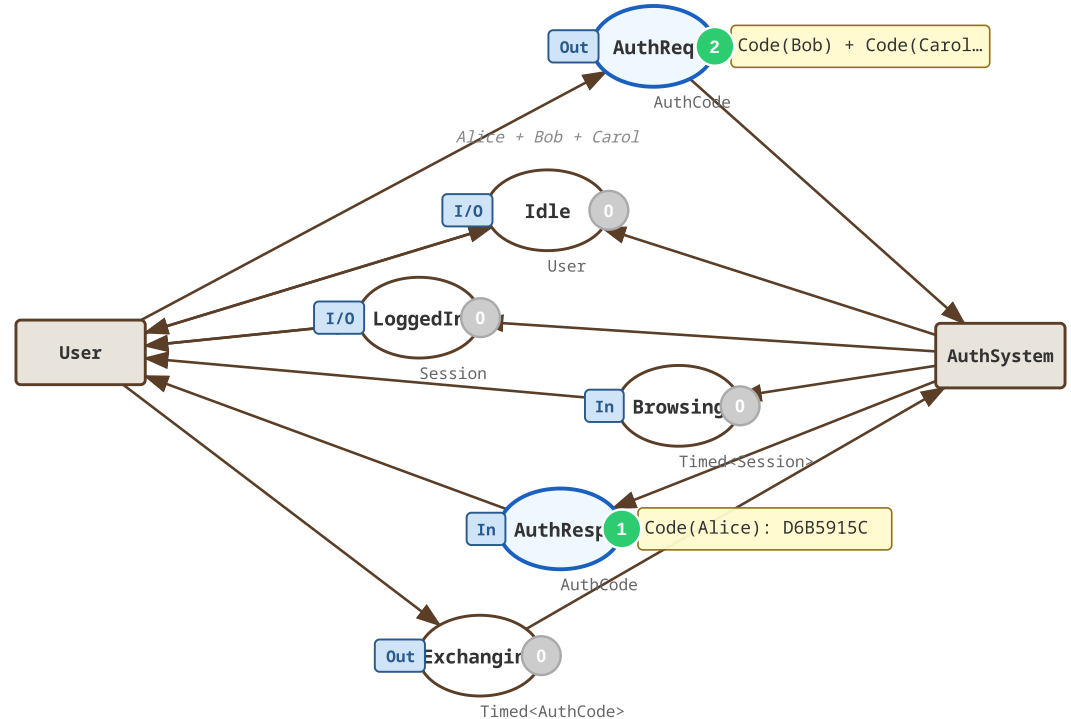


Running example: OAuth2 authoris

Exercises all three features:

- Three concurrent users
- Shared places as channels
- Timed auth, exchange, expiry
- Two sub-pages: User & AuthSystem

Top page: shared places link the two substitution transitions.



Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set
- Real cryptography (`SHA256`) inside the model

Colour sets are just C# types

Any `notnull, IEquatable<T>` type works — primitives, enums, records, classes.

```
public record User(int Id, string Name) {
  public string CsrfState  => $"s{Id}";
  public int    BrowseTime => Id * 5;
}

public record AuthCode(User User, string Code, string State) {
  public bool ValidCsrf(User u) => State == u.CsrfState;
  public static AuthCode Create(User u, int seq) {
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes($"{u.Id}:{seq}"));
    return new(u, Convert.ToHexString(hash)[..8], u.CsrfState);
  }
}

public record Session(User User, int TokenId);
```

- **Records** — immutable, with value equality
- Methods carry **domain logic** (`CsrfState` , `BrowseTime`)
- The **CSRF check** is part of the colour set
- Real cryptography (`SHA256`) inside the model

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag
- `AddTimedPlace<T>` → place of `Timed<T>` tokens

Places & multisets

```
// inside the model constructor
var authReq = AddPlace<AuthCode>("AuthReq");            // empty initial marking

var idle    = AddPlace("Idle", Multiset.Of(
                new User(1,"Alice"), new User(2,"Bob"), new User(3,"Carol")));

var exchg   = AddTimedPlace<AuthCode>("Exchanging");    // timed place
```

- `Place<T>` — the type parameter (`T`) specifies the colour set
- `Multiset<T>` — immutable, strongly typed bag
- `AddTimedPlace<T>` → place of `Timed<T>` tokens

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Arcs are built to and from transitions

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Arcs are built to and from transitions
- Output arc expressions **C# lambdas** (or just a variable)

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Arcs are built to and from transitions
- Output arc expressions **C# lambdas** (or just a variable)
- Guards are **C# lambdas** — compiler-checked

Transitions: a fluent builder

```
var u   = new Var<User>("u");

AddTransition("Login")
  .Input(idle, u).Output(waiting, u)
  .Output(authReq, () ⇒ AuthCode.Create(u.Val), "AuthReq")
  .Build();

AddTransition("ReceiveCode")
  .Input(authResp, c).Input(waiting, u)
  .Guard(() ⇒ c.Val.ValidCsrf(u.Val))
  .TimedOutput(exchg, () ⇒ c.Val, Config.ExchangeTime)
  .Build();
```

- Variables are strongly typed
- Transitions are build fluently
- Arcs are built to and from transitions
- Output arc expressions **C# lambdas** (or just a variable)
- Guards are **C# lambdas** — compiler-checked

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
  .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                           () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
  .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                             () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
  .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                             () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- One transition fires **three** outputs — session, expiry, browse-timer

Timed CPNs

```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
  .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                             () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- One transition fires **three** outputs — session, expiry, browse-timer
- `SessionExpire` consumes the expiry timer when it matures — races with `Logout` (in `UserPage`) for the same session

Timed CPNs

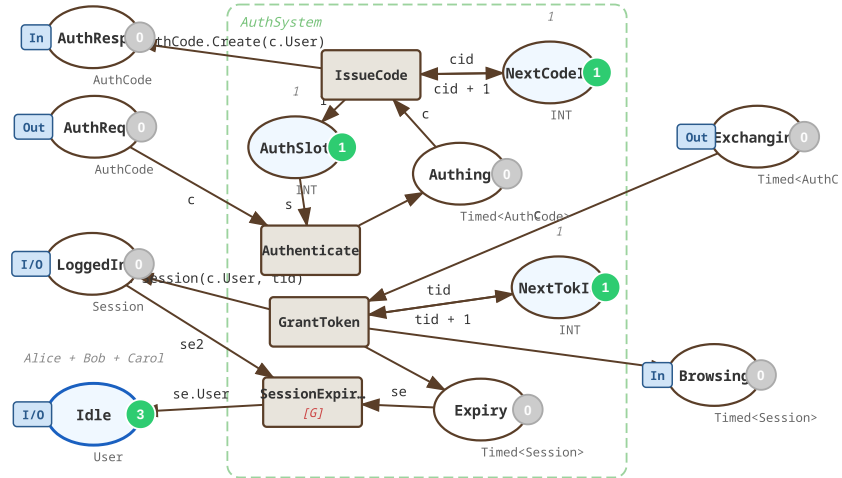
```
AddTransition("GrantToken")
  .TimedInput(exchg, c).Input(nextTokId, tid)
  .Output(nextTokId, () ⇒ tid.Val + 1)
  .Output(loggedIn,  () ⇒ new Session(c.Val.User, tid.Val))
  .TimedOutput(expiry,   () ⇒ new Session(c.Val.User, tid.Val), Config.SessionTTL)
  .TimedOutput(browsing, () ⇒ new Session(c.Val.User, tid.Val),
                             () ⇒ c.Val.User.BrowseTime)
  .Build();

AddTransition("SessionExpire")
  .TimedInput(expiry, se).Input(loggedIn, se2)
  .Guard(() ⇒ se.Val.TokenId == se2.Val.TokenId)
  .Output(idle, () ⇒ se.Val.User)
  .Build();
```

- Global integer clock; `Timed<T>` tokens carry timestamps
- One transition fires **three** outputs — session, expiry, browse-timer
- `SessionExpire` consumes the expiry timer when it matures — races with `Logout` (in `UserPage`) for the same session

Hierarchical CPNs

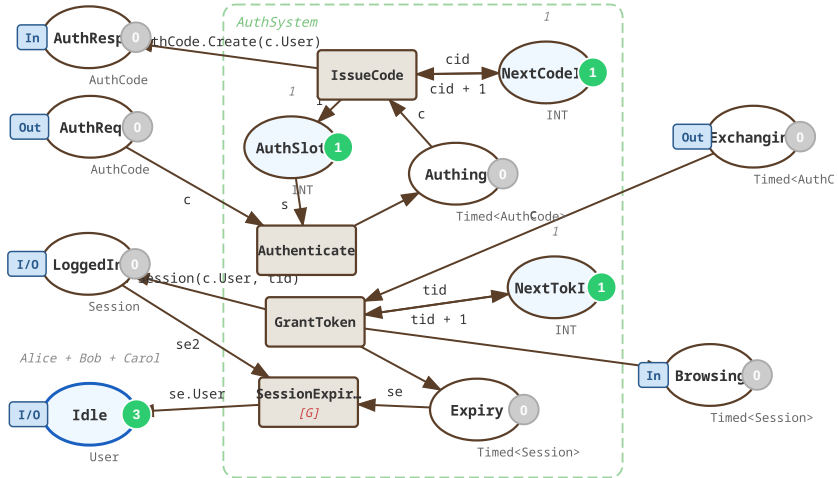
```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
// assembly
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```



AuthSystem sub-page: `AuthSlot` limits concurrency, `Expiry` delays timeout.

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
// assembly
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```



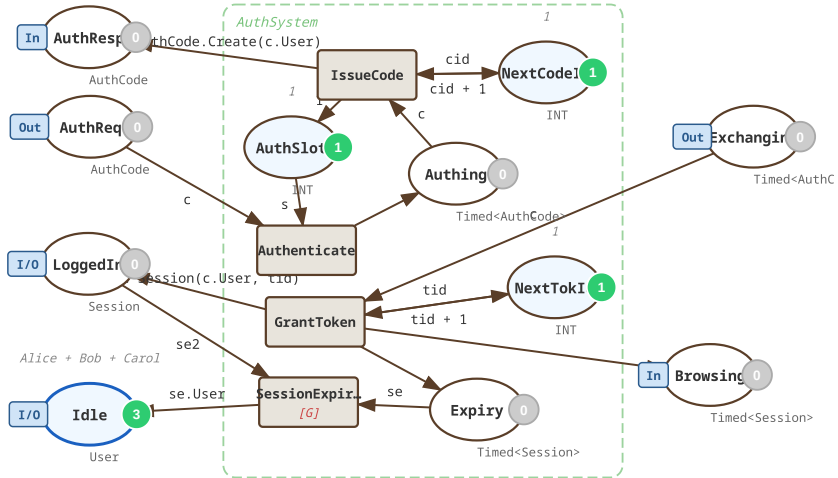
AuthSystem sub-page: `AuthSlot` limits concurrency, `Expiry` delays timeout.

- A sub-page is a `CpnPage` **subclass**

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
// assembly
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

- A sub-page is a `CpnPage` **subclass**
- Ports use `In()` / `Out()` / `InOut()` — CPN Tools style

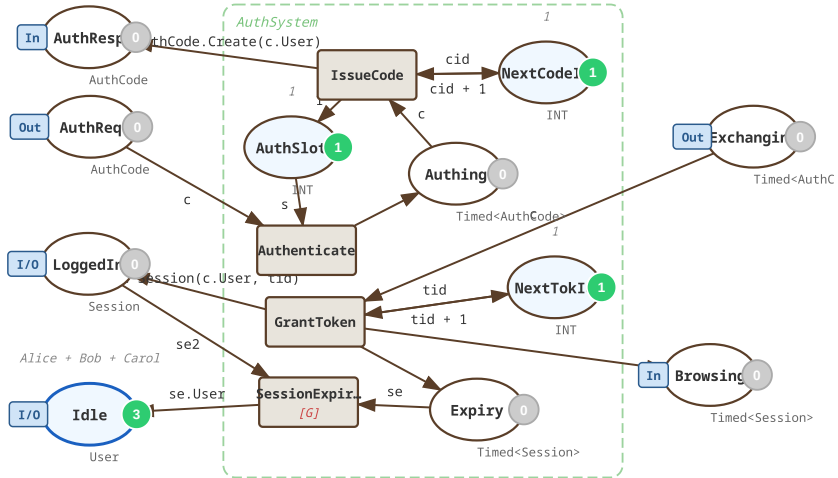


AuthSystem sub-page: `AuthSlot` limits concurrency, `Expiry` delays timeout.

Hierarchical CPNs

```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
// assembly
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

- A sub-page is a `CpnPage` **subclass**
- Ports use `In()` / `Out()` / `InOut()` — CPN Tools style
- `AddSubPage` wires it into the parent

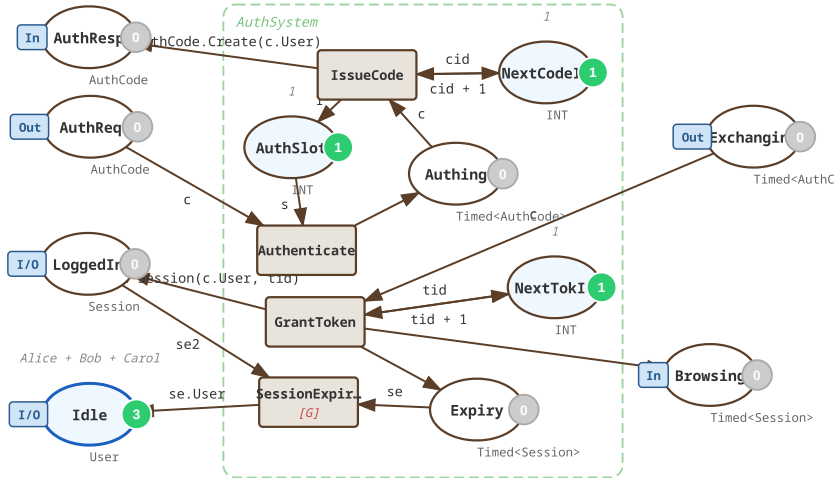


AuthSystem sub-page: `AuthSlot` limits concurrency, `Expiry` delays timeout.

Hierarchical CPNs

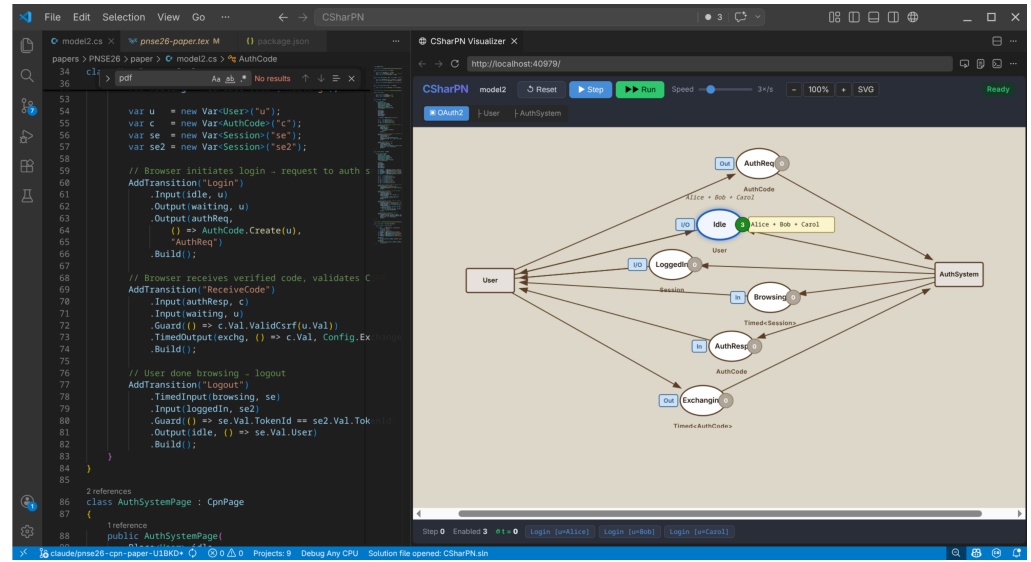
```
class UserPage : CpnPage {
  public UserPage(Place<User> idle,
       Place<AuthCode> authReq, ...)
    : base("User") {
    InOut(idle);
    Out(authReq); In(authResp);
    Out(exchg);
    InOut(loggedIn); In(browsing);
    var waiting = AddPlace<User>("Waiting");
    // Login, ReceiveCode, Logout
  }
}
// assembly
AddSubPage(new UserPage(...),       "User");
AddSubPage(new AuthSystemPage(...), "AuthSystem");
```

- A sub-page is a `CpnPage` **subclass**
- Ports use `In()` / `Out()` / `InOut()` — CPN Tools style
- `AddSubPage` wires it into the parent



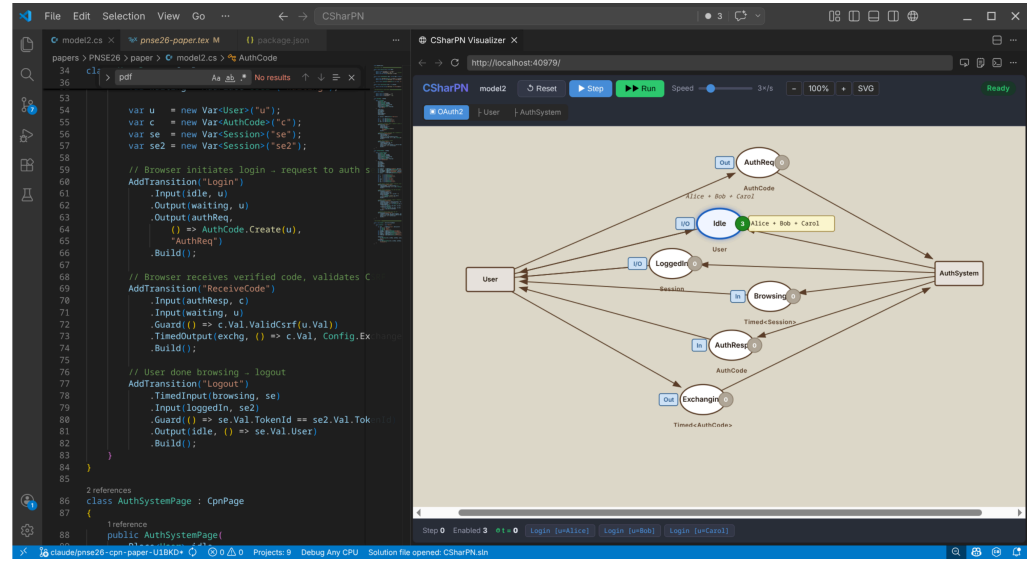
AuthSystem sub-page: `AuthSlot` limits concurrency, `Expiry` delays timeout.

Visualiser & VS Code integration



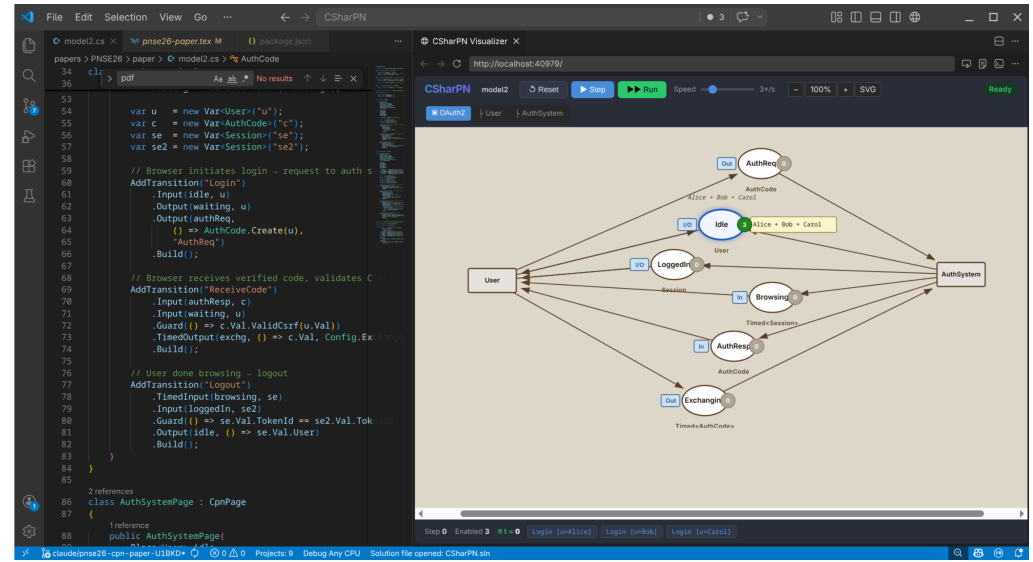
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel



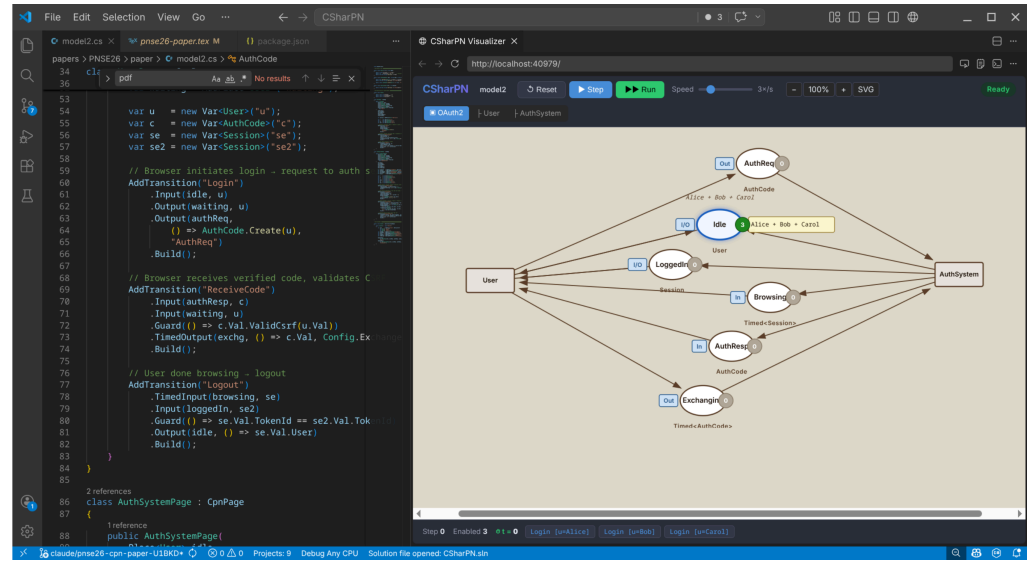
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages



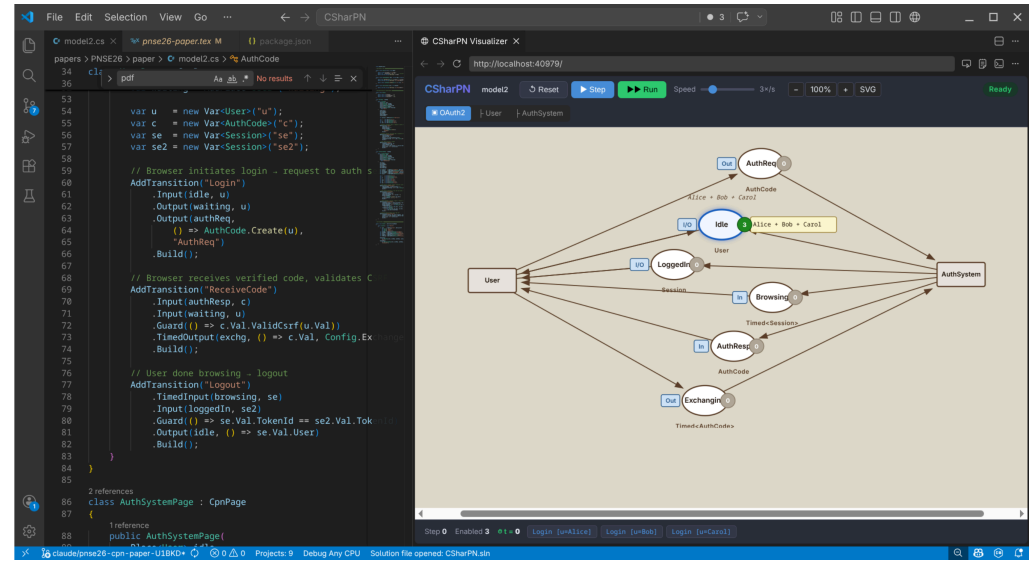
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings



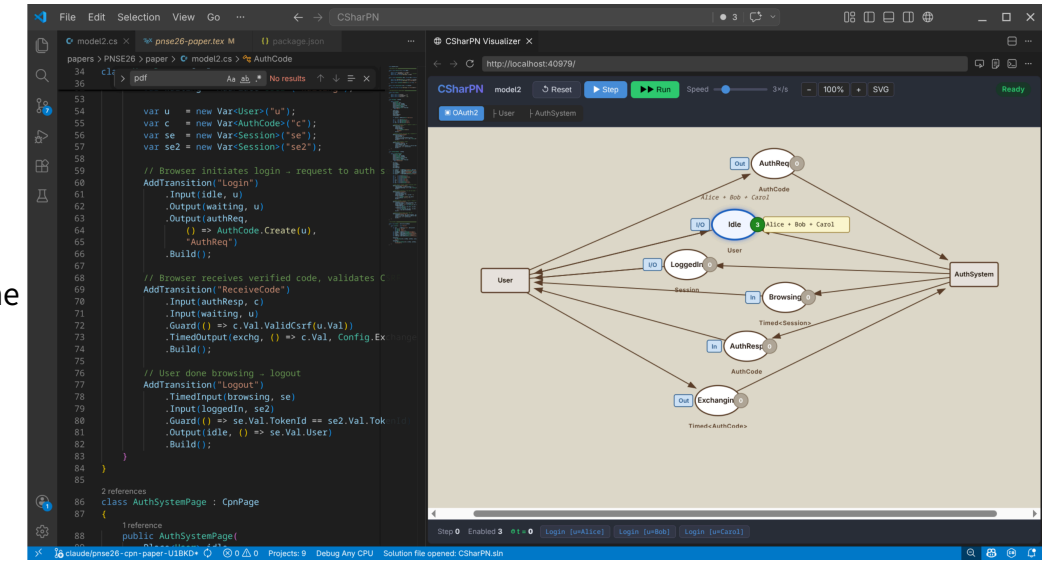
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it



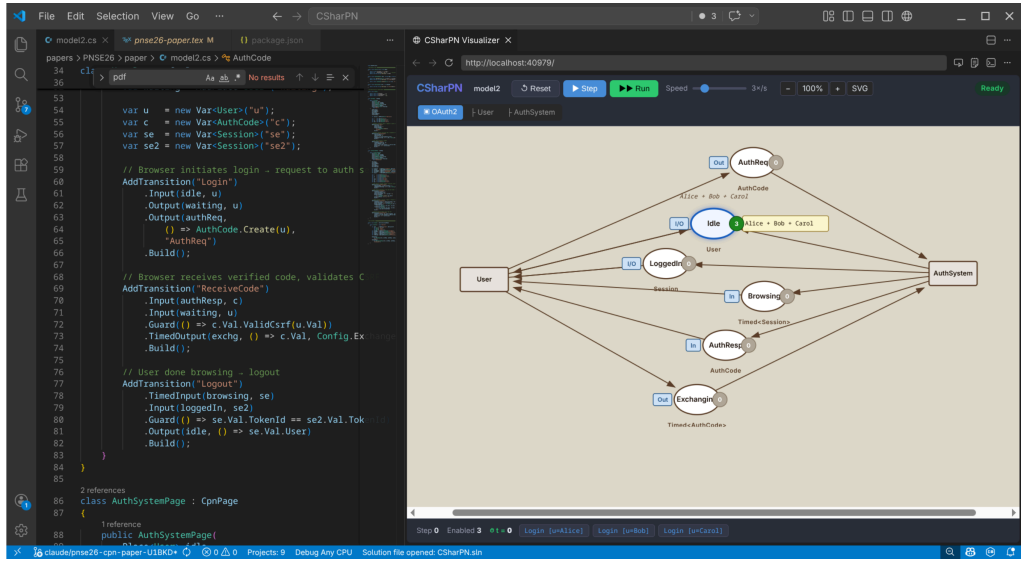
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line



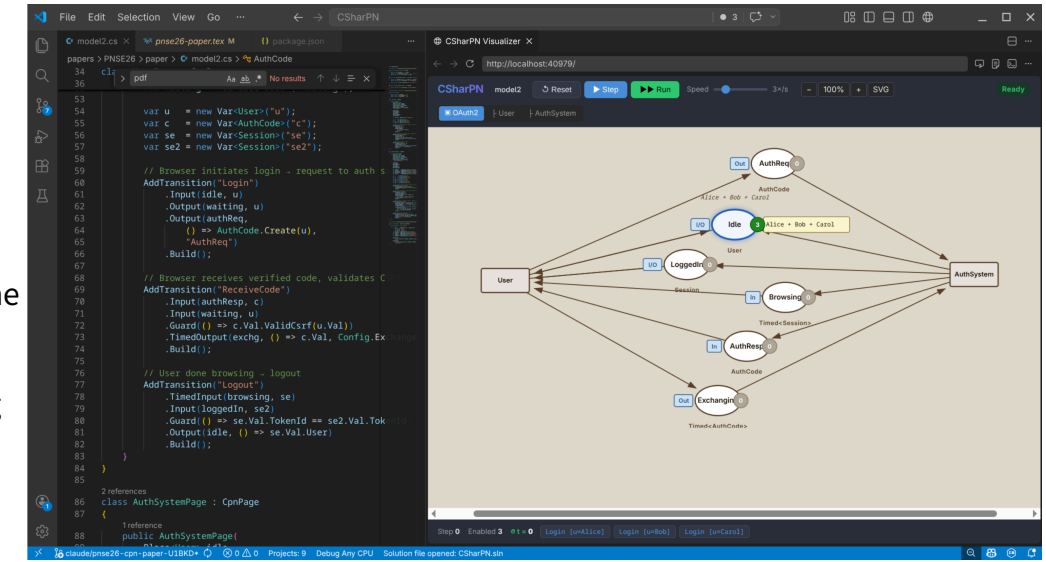
Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line
- Layout persisted in `model.cs.layout`



Visualiser & VS Code integration

- Blazor app in VS Code's Simple Browser panel
- Page tree — drill into sub-pages
- Clock, step count, enabled bindings
- Click a binding to fire it
- **Double-click** a place/transition → jump to source line
- Layout persisted in `model.cs.layout`
- **Hot reload**: save .cs → recompile → reload, marking preserved



Models as programs

key insight

When the inscription language is the host language, the model *is* the program.

Models as programs

key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs

Models as programs

key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs
- **Refactoring & IntelliSense** — rename a colour set across the model

Models as programs key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs
- **Refactoring & IntelliSense** — rename a colour set across the model
- **Unit tests** — xUnit/NUnit asserts on markings

Models as programs key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs
- **Refactoring & IntelliSense** — rename a colour set across the model
- **Unit tests** — xUnit/NUnit asserts on markings
- **Domain libraries** — real crypto, real currency conversion, in your model

Models as programs

key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs
- **Refactoring & IntelliSense** — rename a colour set across the model
- **Unit tests** — xUnit/NUnit asserts on markings
- **Domain libraries** — real crypto, real currency conversion, in your model
- **Composition with production code** — ship the model as a NuGet package

Models as programs key insight

When the inscription language is the host language, the model *is* the program.

- **Version control** — plain `.cs` files, real diffs, real PRs
- **Refactoring & IntelliSense** — rename a colour set across the model
- **Unit tests** — xUnit/NUnit asserts on markings
- **Domain libraries** — real crypto, real currency conversion, in your model
- **Composition with production code** — ship the model as a NuGet package

> **No code-generation step.** No generated artefact, no model/implementation sync problem — the gap is zero by construction. (Caveat: only when the deployment target is .NET.)

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)

- CPN-as-test-harness for existing C# libraries
- State-space analysis on `CpnState` snapshots
- Empirical evaluation: usability study + simulator performance vs CPN Tools

Future work

- Models as deployable .NET components (HTTP-driven transitions/places)

- CPN-as-test-harness for existing C# libraries
- State-space analysis on `CpnState` snapshots
- Empirical evaluation: usability study + simulator performance vs CPN Tools

> **Takeaway:** when the inscription language is C#, the full power of a mature programming language — type system, libraries, tooling — is available to the CPN modeller at no extra cost.

Thank you

Questions?

CSharPN — MIT licensed

`github.com/simontjell/CSharPN.public`

Simon Tjell · simon@simplesystemer.dk